MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 1
Software Design

FitCore

Fitness Management Application

Spring Boot 3 + Angular 17 Architecture

Author: Robert-Daniel Gavrila
Group: 30433

Faculty of Automation and Computer Science

March 23, 2026

Contents

1 Problem definition **2**

2 Analysis phase - Use cases & requirements **3**
2.1 Functional requirements . . . 3
2.2 Non-Functional Requirements . . . 3
2.3 Use cases . . . 4
2.3.1 UC1: User Registration . . . 4
2.3.2 UC2: User Login . . . 4
2.3.3 UC3: Browse Workouts . . . 4
2.3.4 UC4: Create New Workout . . . 4
2.3.5 UC5: Update Workout . . . 4
2.3.6 UC6: Delete Workout . . . 5
2.3.7 UC7: Generate Diet Plan . . . 5
2.3.8 UC8: Manage Users (Admin Only) . . . 5

3 Design phase **7**
3.1 Architecture overview . . . 7
3.2 Entity-relationship diagram . . . 7
3.3 Backend Component: Spring Boot REST API . . . 9
3.4 Frontend Component: Angular 17 SPA . . . 9
3.5 Design patterns . . . 9
3.6 Class diagrams . . . 9
3.7 Sequence diagrams . . . 10

4 Implementation phase - Application **12**
4.1 Tools and technologies . . . 12
4.2 Application structure . . . 12
4.3 User manual . . . 12

5 Testing and validation **13**

1 Problem definition

The main objective of this project was to introduce us to the early stage of application development, while polishing our coding skills and implementing the design of our software product.

The requirement was to implement a **MVC (Model-View-Controller)** based application, which is a common architectural pattern in software products. It enhances layer separation into three main parts (conceptually, because in real web applications, we should divide it into multiple layers, such as entities, services, controllers, etc.), which suits the **Waterfall** methodology great.

I have identified a common problem in our days: we lack sportive movements and our jobs are moving from physical activities to intellectual ones, which comes with a huge drawback: a **sedentary lifestyle**. This affects thousands of persons annually, so with this problem in mind, I came up with the idea of a software solution.

FitCore is a comprehensive fitness management application designed to help users track their workouts, plan their diets, and maintain a healthy lifestyle. The application provides an intuitive interface for creating and managing fitness routines, while offering administrators powerful tools to manage the platform and its users.

As the **Waterfall** methodology is too rigid for today's fast movements from one requirement to another, and because we are building software products for a large range of people, we must consider maintainability and extension features while assuring scalability and performance of our product.

As a development methodology, I have respected **Agile Methodologies**, building a **Kanban** table where all key features were clearly pointed out in issues, which were assigned to the development team (for this assignment, just me), simulating a real development phase.

As deliverables, I have wrapped up my web-based application in a Github repository, consisting of a **Spring Boot backend** and an **Angular frontend**, pinpointing the cutting-edge technologies used in real development projects.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

Functional requirements describe the operations that the system must perform. These requirements define the services the system provides and the expected system behavior. The following functional requirements have been identified for the FitCore application:

- **Authentication & Authorization:** The system must allow users to register new accounts and authenticate using email and password credentials. Authentication must use JSON Web Tokens (JWT) for stateless session management.
- **Workout Management:** The system must allow authenticated users to create, read, update, and delete workout records. Each workout must include name, description, muscle group, difficulty level, duration, and estimated calories burned.
- **Workout Filtering:** The system must provide filtering capabilities by name, muscle group, difficulty level, and duration range to help users discover relevant workouts.
- **Sorting & Pagination:** The system must support multi-column sorting (by name, date created, duration, calories) in both ascending and descending order, and implement pagination for efficient data retrieval.
- **Diet Planning:** The system must provide diet plan generation based on user calorie requirements and fitness goals. It must also offer nutrition information lookup for food items.
- **User Management:** Only administrators can view all users, modify user roles, enable/disable accounts, and delete users.
- **Role-Based Access Control:** The system must implement three distinct roles: **VISITOR** (unauthenticated), **USER** (authenticated regular users), and **ADMIN** (administrators with full platform access).
- **Workout Statistics:** The system must provide aggregated statistics about workouts including total count, average duration, and other useful metrics.

2.2 Non-Functional Requirements

- **Security:** User passwords must be hashed using BCrypt with a strength factor of 10. All sensitive operations must require appropriate authentication and authorization checks.
- **Performance:** The application should handle pagination efficiently and support filtering without noticeable performance degradation. Database indexing on frequently queried columns (muscle group, difficulty, creator) is essential.
- **Scalability:** The application must support both in-memory (H2) and production (PostgreSQL) databases. The stateless JWT authentication allows horizontal scaling without session replication concerns.

- **Maintainability:** The codebase follows clean architecture principles with clear separation of concerns: controllers, services, repositories, and entities. Each layer has distinct responsibilities and can be modified independently.

2.3 Use cases

Use cases describe in text the interactions between actors and the system in order to accomplish specific tasks. Each use case includes the actor initiating the interaction, a description of the task being performed, and the result of a successful interaction.

2.3.1 UC1: User Registration

- **Actor:** Visitor (unauthenticated user)
- **Description:** A visitor wants to create a new account by providing their first name, last name, email, and password.
- **Result:** User account is created and the user can now log in to the application.

2.3.2 UC2: User Login

- **Actor:** Registered user or administrator
- **Description:** A user wants to authenticate into the system using their email and password.
- **Result:** User is authenticated and can access protected resources. A JWT token is automatically included in all subsequent API requests.

2.3.3 UC3: Browse Workouts

- **Actor:** Any user (VISITOR, USER, or ADMIN)
- **Description:** User wants to view all available workouts with optional filtering and sorting.
- **Result:** User sees a filtered and sorted workout list matching their criteria.

2.3.4 UC4: Create New Workout

- **Actor:** Authenticated user (USER or ADMIN)
- **Description:** User wants to create a new workout record by providing workout details.
- **Result:** New workout is created and visible in the system.

2.3.5 UC5: Update Workout

- **Actor:** Authenticated user (USER or ADMIN)
- **Description:** User wants to modify an existing workout record.
- **Result:** Workout is updated in the database with new information.

2.3.6 UC6: Delete Workout

- **Actor:** Administrator
- **Description:** Administrator wants to remove a workout from the system.
- **Result:** Workout is permanently deleted from the system.

2.3.7 UC7: Generate Diet Plan

- **Actor:** Authenticated user (USER or ADMIN)
- **Description:** User wants to generate a personalized diet plan based on their calorie needs and fitness goal.
- **Result:** Personalized diet plan is displayed with food recommendations.

2.3.8 UC8: Manage Users (Admin Only)

- **Actor:** Administrator
- **Description:** Administrator wants to view all users and manage their roles and status.
- **Result:** User roles and statuses are updated in the system.

Use Case Diagram:

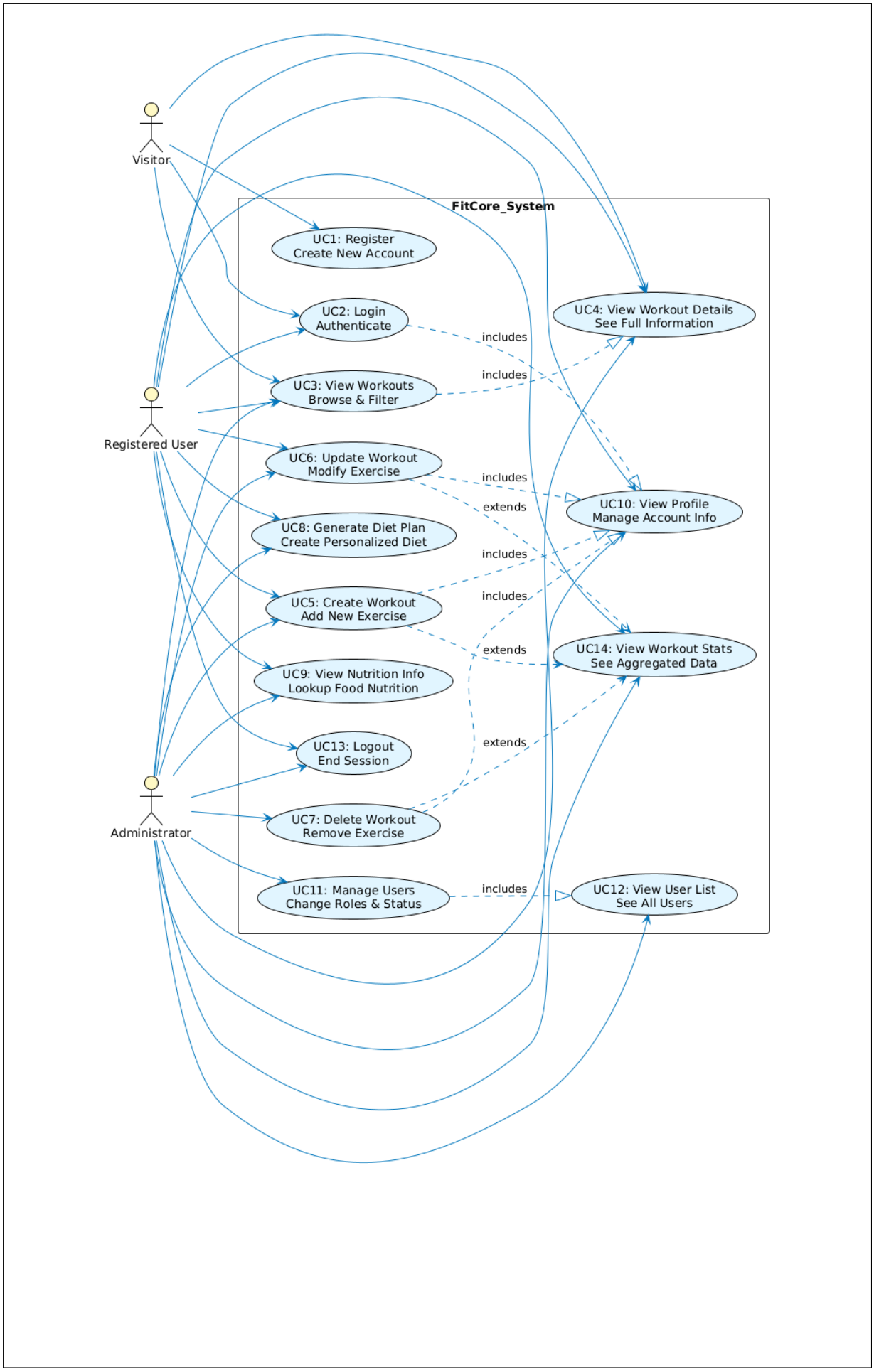


Figure 1: FitCore Use Case Diagram

3 Design phase

3.1 Architecture overview

FitCore implements a **modern three-tier client-server architecture** with clear separation of concerns:

- **Client Layer:** Angular 17 single-page application (SPA) running in the browser
- **API Gateway & Server Layer:** Spring Boot 3 REST API with Spring Security
- **Data Layer:** Persistence layer with Hibernate ORM and JPA
- **Database Layer:** H2 (development) or PostgreSQL (production)

The architecture employs the **Layered Architecture** pattern, which provides:

- **Advantages:** Clear separation of concerns, easy to test with mocks, simple to understand, easy to maintain, and supports role-based development.
- **Disadvantages:** Potential performance impact from multiple layer traversals, can lead to anemic domain models, and may require boilerplate code for simple operations.

3.2 Entity-relationship diagram

This subsection describes the database schema used by the application. The entity-relationship diagram illustrates the structure of the database and the relationships between entities.

Users Table:

- **Purpose:** Stores user account information and authentication credentials.
- **Important attributes:** `id`, `email`, `password`, `role`.
- **Relationships:** One-to-Many with Workouts (creator); One-to-Many with UserWorkoutLogs.

Workouts Table:

- **Purpose:** Stores workout exercises with their metadata and parameters.
- **Important attributes:** `name`, `muscleGroup`, `difficulty`, `createdBy_id`.
- **Relationships:** Many-to-One with Users; One-to-Many with UserWorkoutLogs.

UserWorkoutLogs Table:

- **Purpose:** Tracks when users perform workouts for activity logging and statistics.
- **Important attributes:** `logDate`, `actualDurationMinutes`.
- **Relationships:** Many-to-One with Users; Many-to-One with Workouts.

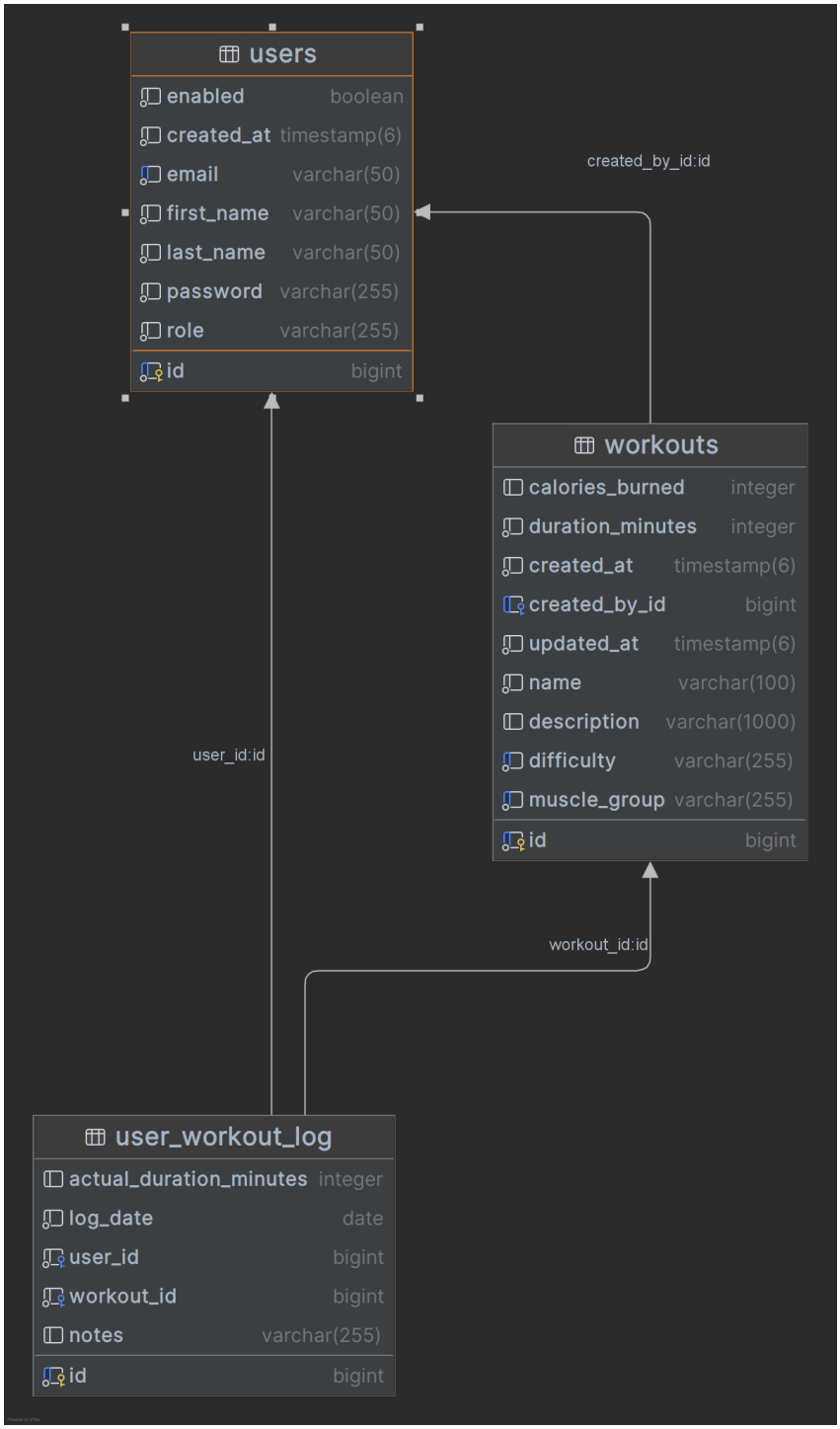


Figure 2: FitCore Database Entity-Relationship Diagram

3.3 Backend Component: Spring Boot REST API

The backend is implemented as a RESTful service using Spring Boot 3 with Spring Data JPA, providing a stateless, scalable API for the Angular frontend.

The responsibilities of this component include handling HTTP requests via Controllers, managing JWT-based authentication using Spring Security, executing business logic within Services, and persisting data via Repositories using Hibernate ORM. It interacts closely with the database layer and serves JSON responses to the client application.

3.4 Frontend Component: Angular 17 SPA

The frontend is built as a modern single-page application using Angular 17 with standalone components, providing a responsive and interactive user interface.

It is responsible for state management, client-side routing, displaying intuitive forms, and communicating with the backend API. It utilizes an `auth.interceptor.ts` to securely append JWT tokens to HTTP requests and implements route guards to protect authenticated views.

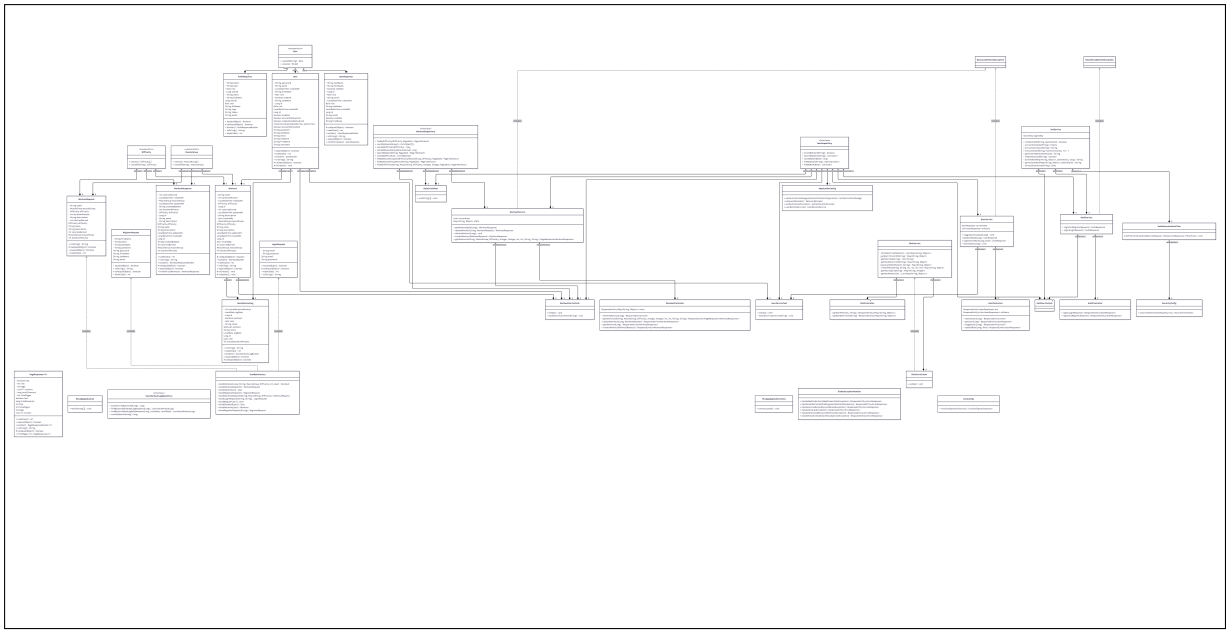
3.5 Design patterns

The FitCore application implements several well-established design patterns to solve recurring problems and improve code quality:

- **Builder Pattern (Creational):** Solves the problem of cumbersome object instantiation for objects with many optional attributes. Implemented using Lombok's `@Builder` on DTOs and entities.
- **Singleton Pattern (Creational):** Ensures services have only one instance per application. Implemented via Spring's `@Service` annotation.
- **Facade Pattern (Structural):** Hides complex business logic behind a simple interface. The Service layer acts as a facade for the Controller layer.
- **Data Transfer Object / DTO Pattern (Structural):** Prevents tight coupling of the API to the database schema and hides sensitive data. Implemented via `WorkoutRequest`, `WorkoutResponse`, etc.
- **Strategy Pattern (Behavioral):** Allows the password encoding strategy to be interchangeable. Implemented using Spring Security's `PasswordEncoder` interface.

3.6 Class diagrams

Class diagrams describe the static structure of the system by presenting classes, attributes, methods, and relationships.



FitCore Backend Layered Architecture Class Diagram

3.7 Sequence diagrams

Sequence diagrams illustrate how objects interact during the execution of a particular use case.

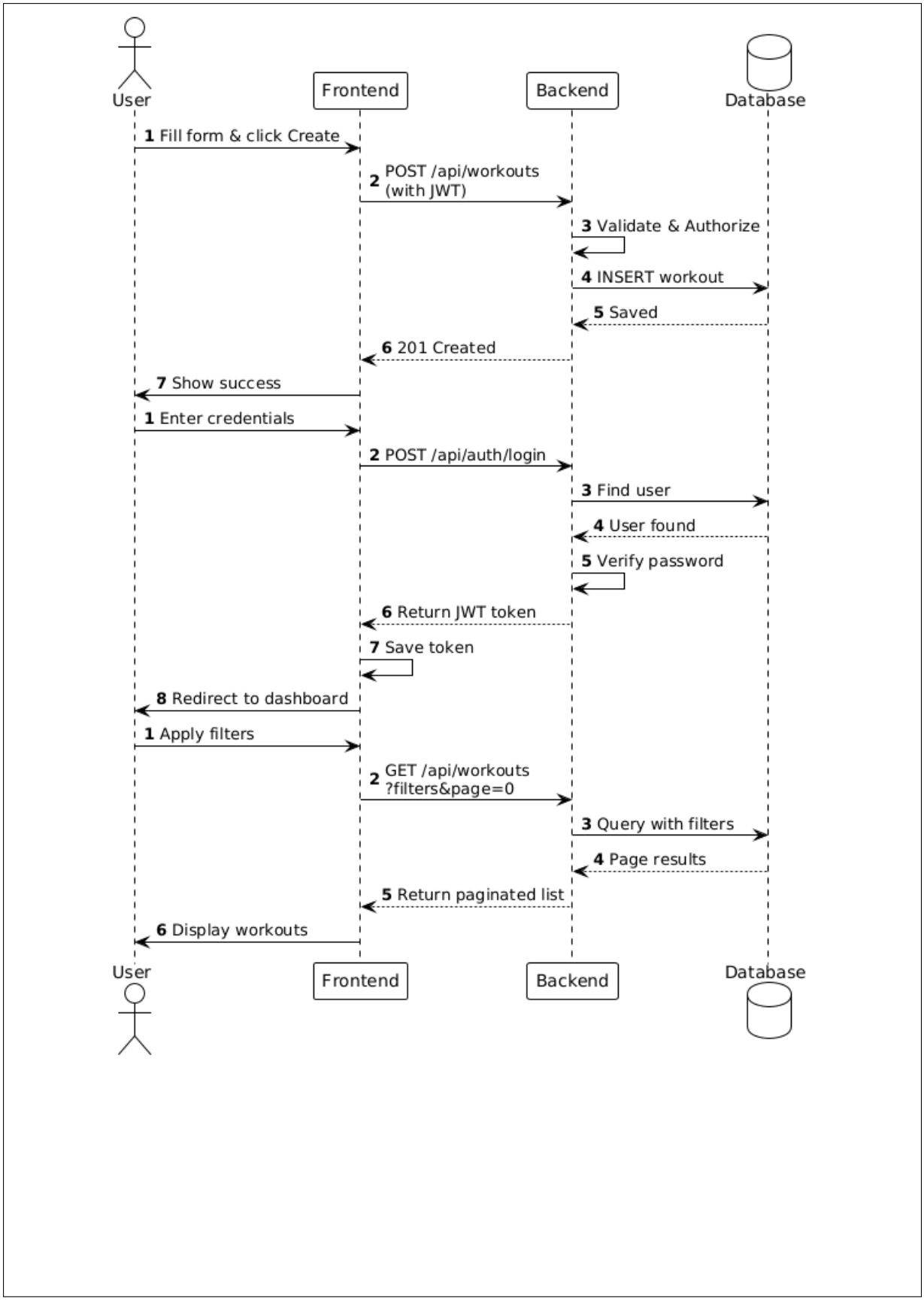


Figure 3: Workout Filtering and Pagination Sequence

4 Implementation phase - Application

4.1 Tools and technologies

This subsection presents the technology stack used to implement the application.

- **Programming Languages:** Java 21 (Backend) and TypeScript 5.4 (Frontend).
- **Database Management System:** PostgreSQL (Production) and H2 (Development).
- **Frameworks & Libraries:** Spring Boot 3.2, Spring Security 6.x, Angular 17.3, RxJS.
- **Communication Protocols:** REST over HTTP/HTTPS utilizing JSON payloads and JWT for stateless authentication.

These technologies were chosen because Spring Boot 3 paired with Java 21 offers advanced features like virtual threads and robust security, while Angular 17's standalone components significantly reduce boilerplate code, resulting in a highly scalable and maintainable enterprise-grade application.

4.2 Application structure

The backend code is organized into discrete packages representing architectural layers: `controllers`, `services`, `repository`, `entity`, `dto`, and `security`. The frontend is organized following Angular best practices into `core` (services, guards, interceptors), `features` (dashboard, workouts, auth), and `shared` modules.

4.3 User manual

The graphical user interface provides specific views depending on the user's role:

Anonymous Users (Visitors) can browse and filter the workout list, view workout details, and access the registration and login screens.

Registered Users gain access to a personalized dashboard where they can create and edit their own workouts, generate custom diet plans based on caloric needs, and view nutritional information.

Administrators have access to a dedicated Admin Panel allowing them to change user roles, disable accounts, and permanently delete workouts or users.

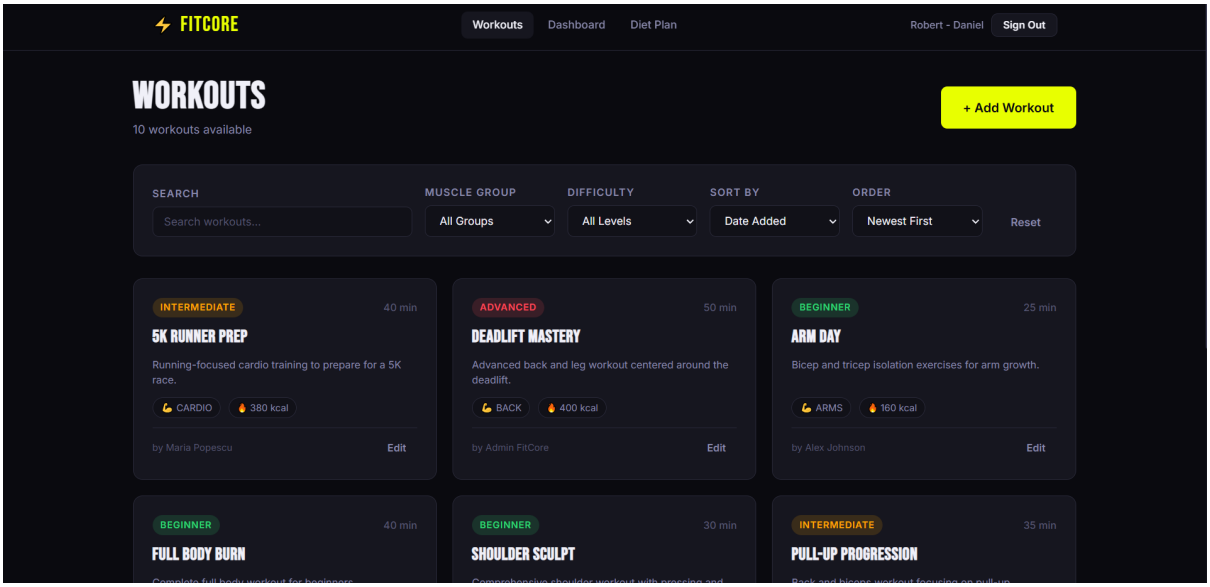


Figure 4: Workout List with Filters

5 Testing and validation

This section describes the methods used to verify that the application works correctly.

The testing strategy focuses heavily on unit testing the service layer, which contains the core business logic. Testing is implemented using **JUnit 5** and **Mockito**.

For example, the `WorkoutService` is comprehensively tested to validate retrieval logic (handling filters, sorting, and pagination correctly), creation and update rules (timestamp validations, creator associations), and deletion. The test coverage ensures that the implemented system perfectly satisfies the requirements defined in the analysis phase.

The project is built to support CI/CD pipelines, utilizing Maven to enforce compilation and test success before artifacts are generated.